

[75.07 / 95.02 / TB025]

Algoritmos y programación III

Paradigmas y programación

Trabajo práctico 2: Mafia

Estudiantes:

Nombre	Padrón	Mail

Tutor:

Nota Final:

Índice

1. Objetivo	4
2. Consigna general	4
3. Especificación de la aplicación a desarrollar	4
3.1. Objetivo General del Juego	4
3.2. Componentes y Estructura del Mazo	4
3.3. Jugadores y Estructura de la Partida	5
A. Inicio de la Partida (Setup)	5
B. Estructura de la Ronda	5
Fase Nocturna	5
Fase Diurna	6
3.4. Roles Especiales:	6
3.5. Condiciones de Victoria	7
3.6. Registro de Estado y Transparencia	7
4. Interfaz gráfica	8
5. Herramientas	8
6. Entregables	9
7. Formas de entrega	9
8. Evaluación	9
9. Entregables para cada fecha de entrega	10
Entrega 0 (Semana 13 del calendario)	10
Entrega 1 (Semana 14 del calendario)	10
Entrega 2 (Semana 15 del calendario)	11
Entrega 3 (Semana 16 del calendario)	11
Entrega 4 (A definir con el corrector)	11
10. Informe	12
Supuestos	12
Diagramas de clases	12
Diagramas de secuencia	12
Diagrama de paquetes	12
Detalles de implementación	12
Excepciones	12
Anexo 0: ¿Cómo empezar?	13
Requisitos, análisis	13
Anexo I: Buenas prácticas en la interfaz gráfica	13
Prototipo	13
JavaFX	13
Recomendaciones visuales	14
Tamaño de elementos	14
Contraste	14
Uso del color	14
Tipografía	14
Recomendaciones de interacción	15
Manejo de errores	15
Confirmaciones	15

1. Objetivo

Desarrollar una aplicación de manera grupal aplicando todos los conceptos vistos en el curso, utilizando un lenguaje de tipado estático (Java) con un diseño del modelo orientado a objetos y trabajando con las técnicas de TDD e Integración Continua.

2. Consigna general

Desarrollar la aplicación completa, incluyendo el modelo de clases, sonidos e interfaz gráfica. La aplicación deberá ser acompañada por pruebas unitarias e integrales y documentación de diseño.

3. Especificación de la aplicación a desarrollar

3.1. Objetivo General del Juego

Mafia es un juego de deducción social por turnos en el que dos bandos se enfrentan: la Mafia (minoría) y los Ciudadanos (mayoría). La partida transcurre en rondas que alternan una Fase Nocturna, donde la Mafia actúa en secreto, y una Fase Diurna, donde todos los jugadores debaten y votan para eliminar a un sospechoso. La Mafia gana cuando iguala o supera en número a los Ciudadanos vivos. Los Ciudadanos ganan cuando eliminan a todos los miembros de la Mafia.

3.2. Componentes y Estructura del Mazo

El juego utiliza un mazo de cartas de rol que se reparten de forma aleatoria y en secreto al inicio de cada partida. Cada jugador recibe exactamente una carta que define su bando y su habilidad especial (si posee alguna). Las cartas no se revelan hasta que el jugador es eliminado o hasta que el juego concluye.

Bando	Rol	Cantidad	Descripcion
Mafia	Mafioso	Variable (1 por cada 3-4 jugadores)	Conoce la identidad de sus cómplices. Durante la noche, la Mafia elige en conjunto a un Ciudadano para eliminar.
Mafia	Padrino	1 (opcional, reemplaza a un Mafioso)	Líder de la Mafia. Aparece como Inocente ante la investigación del Detective. Decide el voto final de la Mafia en la Fase Nocturna.
Ciudadanos	Ciudadano	Variable (mayoria)	No tiene habilidad especial. Solo puede actuar durante el día mediante el debate y el voto.
Ciudadanos	Detective	1 (opcional)	Durante la noche puede investigar a un jugador y conocer su bando (Mafia o Ciudadano). El Padrino aparece como Ciudadano.

Ciudadanos	Medico	1 (opcional)	Durante la noche puede proteger a un jugador (incluso a sí mismo). Si la Mafia elige eliminar a ese jugador, la eliminación es anulada.
Ciudadanos	Sheriff	1 (opcional)	Durante el día puede revelar públicamente el resultado de una investigación previa, ganando credibilidad pero exponiendo su rol.

La composición del mazo se ajusta según la cantidad de jugadores para garantizar el balance entre bandos. Como referencia orientativa:

Jugadores	Mafiosos	Roles especiales sugeridos
5 - 6	1 - 2	Detective o Médico (1 rol especial)
7 - 9	2 - 3	Detective + Médico
10 - 12	3	Detective + Médico + Padrino + Sheriff

3.3. Jugadores y Estructura de la Partida

El juego soporta de 5 a 12 jugadores. Uno de ellos actuará como Moderador (también llamado Narrador): no recibe carta de rol, no participa en el juego como jugador activo y es el responsable de conducir la partida, anunciar los eventos y garantizar que la información secreta no se filtre. En la aplicación, el rol de Moderador es asumido por el sistema.

A. Inicio de la Partida (Setup)

1. Configuración: El sistema solicita la cantidad de jugadores y genera automáticamente la composición del mazo de forma balanceada (o permite configuración manual avanzada).
2. Reparto de cartas: El sistema asigna aleatoriamente una carta de rol a cada jugador. Cada jugador solo puede ver su propia carta.
3. Reconocimiento nocturno inicial: Antes de la primera ronda, el sistema conduce una fase de apertura en la que los Mafiosos conocen entre si la identidad de sus cómplices (sin que los demás jugadores lo vean).

B. Estructura de la Ronda

Cada ronda se divide en dos fases que se alternan hasta que se cumpla una condición de victoria:

Fase Nocturna

Durante la noche todos los jugadores cierran los ojos (en la aplicación, la pantalla se oculta o cada jugador actúa en su dispositivo). El sistema activa los roles en el siguiente orden fijo:

4. Mafia: Los Mafiosos (y el Padrino, si existe) seleccionan en secreto a un jugador vivo para eliminar. Deben llegar a un consenso; si no lo hacen, el sistema puede aplicar mayoría o declarar la noche sin víctima, según la configuración elegida.
5. Detective (si está vivo): El Detective elige a un jugador vivo para investigar. El sistema le revela en privado si ese jugador pertenece a la Mafia o a los Ciudadanos (el Padrino aparece como Ciudadano).

6. Médico (si está vivo): El Médico elige a un jugador (o a sí mismo) para protegerlo durante esa noche. Si la Mafia eligió a ese mismo jugador, la eliminación queda anulada.

Al finalizar la Fase Nocturna, el sistema anuncia públicamente el resultado: si hubo una víctima (y quien fue) o si la noche paso sin eliminaciones (cuando el Médico salvó al objetivo de la Mafia). La carta del jugador eliminado se revela.

Fase Diurna

Con la información del resultado nocturno, todos los jugadores vivos participan de la siguiente secuencia:

7. Debate: Los jugadores discuten libremente, acusan y defienden. En la aplicación se puede implementar como un chat de texto o como un temporizador de debate con un tiempo configurable.
8. Nominaciones: Cada jugador puede nombrar a un sospechoso. Solo los jugadores con al menos una nominación pasan a la fase de votación.
9. Votación: Cada jugador vivo emite un voto para eliminar a uno de los nominados. Gana quien recibe la mayoría simple de votos.
 - En caso de empate: el sistema puede configurarse para aplicar ballotage (segunda vuelta solo entre los empatados) o para declarar que nadie es eliminado en esa ronda.
 - Abstención: Un jugador puede abstenerse de votar; su voto no cuenta para ningún nominado.
10. Eliminación diurna: El jugador más votado es eliminado. Se revela su carta de rol. Si había empate sin mecanismo de desempate, no se elimina a nadie.
11. Verificación de victoria: El sistema comprueba si alguna condición de fin de partida fue alcanzada. Si no, comienza una nueva Fase Nocturna.

3.4. Roles Especiales:

Rol	Bando	Momento de accion	Efecto y restricciones
Mafioso	Mafia	Fase Nocturna	Vota junto a sus complices para elegir una victima. No puede elegir a otro Mafioso. Su carta se revela al ser eliminado.
Padrino	Mafia	Fase Nocturna	Vota con la Mafia y su voto tiene prioridad en caso de desacuerdo. Aparece como Ciudadano ante la investigacion del Detective. Su identidad se revela solo al ser eliminado.
Ciudadano	Ciudadanos	Fase Diurna	Participa en el debate y la votacion. No tiene accion nocturna.
Detective	Ciudadanos	Fase Nocturna	Investiga a un jugador vivo por noche. Recibe el resultado en privado: 'Mafia' o 'Ciudadano'. No puede investigar al mismo jugador dos noches seguidas.
Medico	Ciudadanos	Fase Nocturna	Protege a un jugador por noche. Si ese jugador es el objetivo de la Mafia, la

			eliminacion se anula. No puede proteger al mismo jugador dos noches consecutivas.
Sheriff	Ciudadanos	Fase Diurna	Una vez por partida puede revelar publicamente el resultado de su investigacion (si es Detective) o declarar su rol para ganar autoridad en el debate. Al revelarse, se convierte en objetivo prioritario de la Mafia.

3.5. Condiciones de Victoria

El sistema verifica las condiciones de victoria al final de cada Fase Nocturna y al final de cada Fase Diurna:

Condicion	Bando ganador
El numero de Mafiosos vivos es igual o mayor al numero de Ciudadanos vivos.	Mafia
Todos los Mafiosos han sido eliminados.	Ciudadanos
Solo queda un jugador vivo y es un Mafioso.	Mafia
Solo queda un jugador vivo y es un Ciudadano.	Ciudadanos

Al detectarse una condicion de victoria, el sistema anuncia al bando ganador, revela las cartas de todos los jugadores y muestra un resumen de la partida (historial de eliminaciones, roles revelados, decisiones clave).

3.6. Registro de Estado y Transparencia

El sistema debera mantener en todo momento un registro interno del estado de la partida que incluya:

- Lista de jugadores vivos y eliminados, con sus roles (ocultos para los jugadores, visibles para el sistema).
- Historial de acciones nocturnas: objetivo de la Mafia, investigaciones del Detective y protecciones del Medico por cada ronda.
- Historial de votaciones diurnas: nominaciones y votos de cada jugador por cada ronda.
- Ronda actual y fase activa (Nocturna o Diurna).

Este registro es la base para la verificacion de condiciones de victoria, la validacion de acciones y la generacion del resumen final.

4. Interfaz gráfica

La interacción entre el usuario y la aplicación deberá ser mediante una interfaz gráfica intuitiva. Consistirá en una aplicación de escritorio utilizando JavaFX y se pondrá mucho énfasis y se evaluará como parte de la consigna su usabilidad. *(en el anexo I se explicarán algunas buenas prácticas para armar la interfaz gráfica de usuario o GUI)*

5. Herramientas

1. JDK (Java Development Kit): Versión 1.8 o superior.
2. JavaFX
3. JUnit 5 y Mockito: Frameworks de pruebas unitarias para Java.
4. IDE (Entorno de desarrollo integrado): Su uso es opcional y cada integrante del grupo puede utilizar uno distinto o incluso el editor de texto que más le guste. Lo importante es que el repositorio de las entregas no contenga ningún archivo de ningún IDE y que la construcción y ejecución de la aplicación sea totalmente independiente del entorno de desarrollo.
 - a. [Eclipse](#)
 - b. [IntelliJ](#)
 - c. [Netbeans](#)
5. Herramienta de construcción: Se deberán incluir todos los archivos XML necesarios para la compilación y construcción automatizada de la aplicación. El informe deberá contener instrucciones acerca de los comandos necesarios (preferentemente también en el archivo README.md del repositorio). Puede usarse Maven o Apache Ant con Ivy.
6. Repositorio remoto: Todas las entregas deberán ser subidas a un repositorio único en GitHub para todo el grupo en donde quedarán registrados los aportes de cada miembro. El repositorio puede ser público o privado. En caso de ser privado debe agregarse al docente corrector como colaborador del repositorio.
7. Git: Herramienta de control de versiones
8. Herramienta de integración continua: Deberá estar configurada de manera tal que cada *commit* dispare la compilación, construcción y ejecución de las pruebas unitarias automáticamente. Algunas de las más populares son:
 - a. Travis-CI
 - b. Jenkins
 - c. Circle-CI
 - d. GitHub Actions (recomendado)

Se recomienda basarse en la estructura del [proyecto base](#) armado por la cátedra.

6. Entregables

Para cada entrega se deberá subir lo siguiente al repositorio:

1. Código fuente de la aplicación completa, incluyendo también: código de la prueba, archivos de recursos.
2. Script para compilación y ejecución (Ant o Maven).
3. Informe, acorde a lo especificado en este documento (en las primeras entregas se podrá incluir solamente un enlace a Overleaf o a Google Docs en donde confeccionen el informe e incluir el archivo PDF solamente en la entrega final).

No se deberá incluir ningún archivo compilado (formato .class) ni tampoco aquellos propios de algún IDE (por ejemplo .idea). Tampoco se deberá incluir archivos de diagramas UML propios de alguna herramienta. Todos los diagramas deben ser exportados como imágenes de manera tal que sea transparente la herramienta que hayan utilizado para crearlos.

7. Formas de entrega

Habrán 4 entregas formales que tendrán una calificación de **APROBADO** o **NO APROBADO** en el momento de la entrega. Además, se contará con una entrega 0 preliminar.

Aquel grupo que acumule 2 no aprobados, quedará automáticamente desaprobado con la consiguiente **pérdida de regularidad en la materia de todos los integrantes del grupo**. En cada entrega se deberá incluir el informe actualizado.

8. Evaluación

El día de cada entrega, cada ayudante convocará a los integrantes de su grupo, solicitará el informe correspondiente e iniciará la corrección mediante una entrevista grupal. **Es imprescindible la presencia de todos los integrantes del grupo el día de cada corrección.**

Se evaluará el trabajo grupal y a cada integrante en forma individual. El objetivo de esto es comprender la dinámica de trabajo del equipo y los roles que ha desempeñado cada integrante del grupo. Para que el alumno apruebe el trabajo práctico debe estar aprobado en los dos aspectos: grupal e individual (se revisarán los commits de cada integrante en el repositorio).

Dentro de los ítems a chequear el ayudante evaluará aspectos formales (como ser la forma de presentación del informe), aspectos funcionales: que se resuelva el problema planteado y aspectos operativos: que el TP funcione integrado.

9. Entregables para cada fecha de entrega

Cada entrega consta de las pruebas + el código que hace pasar dichas pruebas.

Cada entrega presupone que los equipos realizarán refactor de su código según crean conveniente.

Entrega 0 (Semana 13 del calendario)

- Planteo de modelo tentativo.
- Diagrama de clases general.
- Diagrama de secuencia para el caso donde el sistema ejecuta una Fase Nocturna completa: la Mafia elige una víctima, el Detective investiga un jugador y el Medico protege a otro, culminando con el cálculo del resultado (víctima eliminada o protegida).

Entrega 1 (Semana 14 del calendario)

Pruebas (sin interfaz gráfica):

- Verificar que el mazo de roles se genere con la composición correcta según la cantidad de jugadores configurada (cantidad correcta de Mafiosos, Ciudadanos y roles especiales).
- Verificar que el reparto de cartas sea aleatorio y que cada jugador reciba exactamente un rol.
- Verificar que un jugador solo pueda ver su propio rol y no el de los demás durante la partida.
- Verificar que los Mafiosos conozcan entre sí la identidad de sus cómplices al inicio de la partida, pero no puedan ver los roles del resto.
- Verificar que la Mafia pueda seleccionar una víctima válida durante la Fase Nocturna (jugador vivo, no Mafioso).
- Verificar que al intentar seleccionar una víctima inválida (jugador muerto o Mafioso) el sistema rechace la acción.
- Verificar que si el Médico protege al mismo jugador que eligió la Mafia, la eliminación quede anulada y el jugador siga vivo.
- Verificar que si la Mafia elige a un jugador no protegido, ese jugador pase al estado eliminado al finalizar la Fase Nocturna.

Entrega 2 (Semana 15 del calendario)

Pruebas (sin interfaz grafica):

- Verificar que el Detective reciba el resultado correcto al investigar a un Mafioso ('Mafia') y a un Ciudadano ('Ciudadano').
- Verificar que el Detective reciba 'Ciudadano' al investigar al Padrino (aunque este sea de la Mafia).
- Verificar que el Detective no pueda investigar al mismo jugador dos noches consecutivas.
- Verificar que el Medico no pueda proteger al mismo jugador dos noches consecutivas.
- Verificar que las nominaciones en la Fase Diurna solo incluyan a jugadores vivos.
- Verificar que la votacion diurna elimine al jugador con mayor cantidad de votos.
- Verificar el mecanismo de empate configurado: ballotage (segunda votacion entre empatados) o declaracion de ronda sin eliminacion.
- Verificar que la carta del jugador eliminado (diurna o nocturnamente) quede revelada para todos al ser eliminado.
- Verificar que un jugador eliminado no pueda realizar ninguna accion en rondas posteriores.
- Planteo inicial de interfaz grafica (mockups/dibujos) mostrando: la pantalla de setup de partida, la vista de Fase Nocturna (con ocultamiento de informacion) y la vista de Fase Diurna con el panel de votacion.

Entrega 3 (Semana 16 del calendario)

- Modelo del juego terminado, incluyendo todos los roles especiales y sus interacciones.
- Verificar las condiciones de victoria: la Mafia gana cuando iguala o supera en numero a los Ciudadanos; los Ciudadanos ganan cuando eliminan a todos los Mafiosos.
- Verificar que el sistema detecte y anuncie correctamente la condicion de victoria al final de cada fase.
- Interfaz grafica inicial basica: configuracion de partida, reparto de roles con pantalla de ocultamiento entre jugadores y visualizacion del estado de la ronda actual.
- Modelo del manejo de rondas: alternancia correcta entre Fase Nocturna y Fase Diurna, con activacion en orden de los roles especiales.

Entrega 4 (A definir con el corrector)

Trabajo Práctico completo funcionando, con interfaz gráfica final, **sonidos** e informe completo.

Tiempo total de desarrollo del trabajo práctico:

5 semanas

10. Informe

El informe deberá estar subdividido en las siguientes secciones:

Supuestos

Documentar todos los supuestos hechos sobre el enunciado. Asegurarse de validar con los docentes.

Diagramas de clases

Varios diagramas de clases, mostrando la relación estática entre las clases. Pueden agregar todo el texto necesario para aclarar y explicar su diseño de manera tal que el modelo logre comunicarse de manera efectiva.

Diagramas de secuencia

Varios diagramas de secuencia, mostrando la relación dinámica entre distintos objetos planteando una gran cantidad de escenarios que contemplen las secuencias más interesantes del modelo.

Diagrama de paquetes

Incluir un diagrama de paquetes UML para mostrar el acoplamiento de su trabajo.

Detalles de implementación

Deben detallar/explicar qué estrategias utilizaron para resolver todos los puntos más conflictivos del trabajo práctico. Justificar el uso de herencia vs. delegación, mencionar que principio de diseño aplicaron en qué caso y mencionar qué patrones de diseño fueron utilizados y por qué motivos.

IMPORTANTE

No describir el concepto de herencia, delegación, principio de diseño o patrón de diseño. Solo justificar su utilización.

Excepciones

Explicar las excepciones creadas, con qué fin fueron creadas y cómo y dónde se las atrapa explicando qué acciones se toman al respecto una vez capturadas.

Anexo 0: ¿Cómo empezar?

Requisitos, análisis

¿Cómo se empieza? La respuesta es lápiz y papel. No código!.

1. Entiendan el dominio del problema. Definir y utilizar un lenguaje común que todo el equipo entiende y comparte. Ej.: Si hablamos de “X entidad”, todos entienden que es algo ... Si los conceptos son ambiguos nunca podrán crear un modelo congruente.
2. Compartan entre el grupo, pidan opiniones y refinan la idea en papel antes de sentarse a programar.

Anexo I: Buenas prácticas en la interfaz gráfica

El objetivo de esta sección es recopilar algunas recomendaciones en la creación de interfaces gráficas de usuario o GUI. La idea no es exigir un diseño refinado y prolijo, sino que pueda ser usado por el grupo de docentes de Algo3 sin impedimentos “lo mejor posible”.

Prototipo

Venimos escribiendo código, integrales y UML todo el cuatrimestre. Me están pidiendo una interfaz gráfica. ¿Cómo se empieza? La respuesta es lápiz y papel. No código!.

1. Armen un dibujo o prototipo en papel (pueden usar google docs o cualquier herramienta también) de todas las “pantallas”. No tiene que ser perfecto.
2. Compartan entre el grupo, pidan opiniones y refinan la idea en papel antes de sentarse a programar.

JavaFX

Entender cómo funciona JavaFX es clave para implementar la GUI correctamente. No subestimen el tiempo que lleva implementar y modificar la UI o interfaz de usuario. Lean todo lo que ofrece y las buenas prácticas de la tecnología. Hay plugins específicos para el IDE, pero por más que usen herramientas WYSIWYG, siempre conviene entender la API para mejorar el código autogenerado que casi nunca es óptimo.

Recomendaciones visuales

Tamaño de elementos

- Se recomienda un tamaño de tipografía de al menos a 10 puntos al mayor contraste negro contra blanco para asegurar la legibilidad, o bien 12 puntos.
- Para los botones o elementos interactivos, el tamaño mínimo del área debería ser de 18x18.
- Extra: Los elementos más importantes deberían ser más grandes y estar en posiciones más accesibles (poner ejemplos)

Contraste

- Aseguren que el texto y los elementos tengan buen contraste y se puedan leer bien
- Se sugiere un contraste cercano a 3 entre textos y fondos para texto grande e imágenes.
- Herramientas para verificar contraste: <https://colourcontrast.cc/>
<https://contrast-grid.eightshapes.com>

Uso del color

- La recomendación es no utilizar más de 3 colores para la UI y los elementos
- En el enunciado se ejemplifica con una paleta accesible, pero pueden usar cualquiera para las fichas
- Accesibilidad: Si usan para las fichas rojo y verde, o verde y azul, aseguren que las personas con daltonismo puedan distinguirlas usando letras o símbolos sobre las mismas.
- Dudas eligiendo paletas? <https://color.adobe.com>

Tipografía

- No se recomienda usar más de 2 tipografías para toda la aplicación
- Asegurarse de que esas tipografías se exporten correctamente en en TP
- Evitar tipografías “artísticas” para texto, menú y botones, ya que dificultan la lectura

Recomendaciones de interacción

Manejo de errores

- No escalar excepciones a la GUI. Es un No absoluto. Enviar a la consola.
- Si muestran errores al usuario, el mensaje de error debe estar escrito sin jerga técnica y permitir al usuario continuar y entender lo que está pasando
- Siempre optar por validar y prevenir errores, a dejar que el usuario ejecute la acción y falle.

Confirmaciones

- Antes de cerrar o ejecutar cualquier operación terminal, una buena práctica es pedir confirmación al usuario (para el alcance del tp no sería necesario)

Visibilidad del estado y otros

- Mostrar el estado en el cual está el juego. Siempre debería estar accesible
- Permitir al usuario terminar o cerrar en cualquier momento